

透過 MediaSession 控制媒體

來源：<https://codelabs.developers.google.com/codelabs/supporting-mediasession?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將展開影片範例，新增對 MediaSession 的支援。這可讓 Android 從應用程式外部控制播放，例如透過 Google 助理、電視的遙控器，或是螢幕鎖定畫面和 Wear OS 的畫面控制選項。

目錄

1. [簡介](#)
2. [開始設定](#)
3. [建立媒體工作階段並管理其狀態](#)
4. [包括播放佇列中項目的中繼資料](#)
5. [自訂動作](#)
6. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2020 年 9 月 9 日

在影片播放時加入 MediaSession 有哪些好處？

媒體工作階段是 Android 平台和媒體應用程式之間的必要連結。這項功能不僅會通知 Android 正在播放媒體，以便將媒體動作轉送至正確的工作階段，還會通知平台正在播放的內容和控制方式。

透過應用程式公開 MediaSession 有許多好處，使用者將能享受這些優點。以下列舉幾個絕佳範例。

Google 助理

使用者可以透過「暫停」、「繼續」和「下一個」等語音指令，輕鬆與應用程式中的媒體互動。你也可以使用媒體的中繼資料，瞭解目前播放的內容。

Android TV

在大螢幕體驗中，Android TV 應用程式可為支援 HDMI-CEC 的電視使用者提供傳統遙控器。系統會將播放/暫停、停止、下一首和上一首按鈕發出的指令轉送至應用程式。

螢幕上的媒體遙控器

自 Android 4.0 (API 級別 14) 起，系統可以存取媒體工作階段的播放狀態和中繼資料。這項功能可讓螢幕鎖定顯示媒體遙控器和藝術作品。這項行為會因 Android 版本而異。

背景媒體

即使播放媒體的應用程式在背景執行，您仍可在上述任一情況下控制媒體。

環境 運算

公開媒體資訊 (包括播放內容和控制方式)，即可在裝置間建立橋樑，讓使用者以各種方式與媒體互動。

建構項目

在本程式碼研究室中，您將擴充現有的 Exoplayer 範例，加入媒體工作階段支援功能。您的應用程式將會：

- 正確反映媒體工作階段的啟用狀態
- 將媒體遙控器轉送至 ExoPlayer
- 將佇列中項目的中繼資料傳遞至媒體工作階段

課程內容

- 媒體工作階段為何能提供更豐富的使用者體驗
- 如何建立媒體工作階段及管理其狀態
- 如何將媒體工作階段連結至 ExoPlayer
- 如何在媒體工作階段中加入播放佇列項目的中繼資料
- 如何新增其他 (自訂) 動作

本程式碼研究室著重於 MediaSession SDK。不會討論無關的概念和程式碼區塊，包括 ExoPlayer 實作的詳細資料，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

軟硬體需求

- 最新版 Android Studio (3.5 以上版本)
 - 具備開發 Android 應用程式的基本知識
-

步驟 2 · 約 3 分鐘

2. 開始設定

從哪裡開始？

我們將從 ExoPlayer 的主要範例開始。這個範例包含具有螢幕上播放控制項的影片，但不會直接使用媒體工作階段。我們可以在這裡深入瞭解並新增這些項目！

取得 ExoPlayer 範例

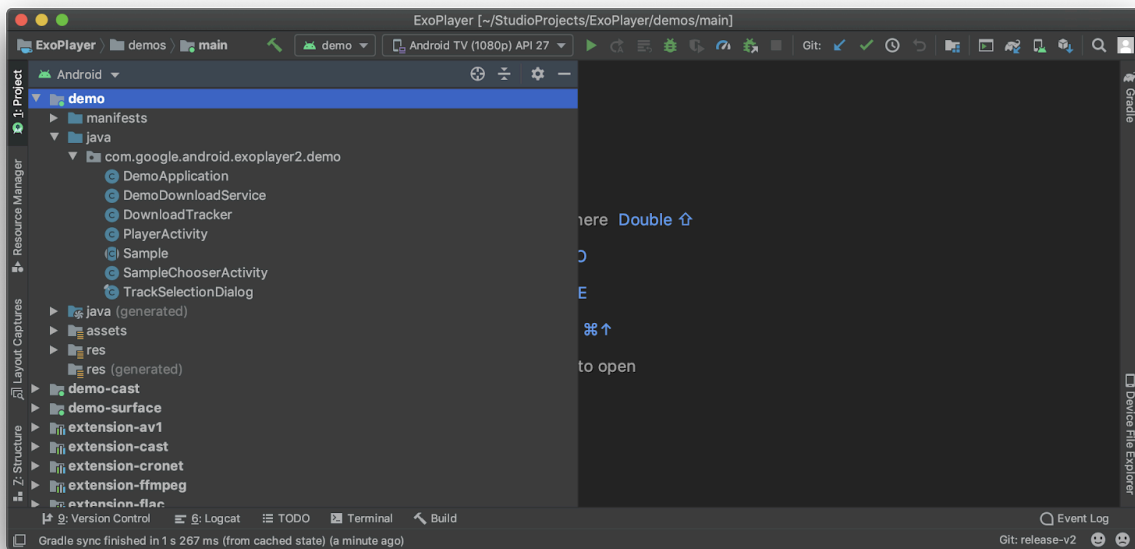
為了快速上手，我們先從 ExoPlayer 範例開始。執行下列程式碼，複製 GitHub 存放區。

```
git clone https://github.com/google/ExoPlayer.git
```

開啟試用版

在 Android Studio 中，開啟 `demos/main` 下的主要範例專案。

Android Studio 會提示您設定 SDK 路徑。如有任何問題，建議您按照「[更新 IDE 和 SDK 工具](#)」中的建議操作。



如果系統要求您使用最新版 Gradle，請更新。

請花點時間基本瞭解應用程式的設計方式。請注意，有兩個活動：`SampleChooserActivity` 和 `PlayerActivity`。我們會在 `PlayerActivity` 中完成其餘的程式碼研究室，媒體實際會在該處播放，因此請開啟這個類別，然後前往下一節。

步驟 3 · 約 10 分鐘

3. 建立媒體工作階段並管理其狀態

建立媒體工作階段

開啟 `PlayerActivity.java`。這個類別會建立 `ExoPlayer`，並管理其函式，例如將影片算繪到螢幕上。在這項活動中，我們會將 `ExoPlayer` 連線至媒體工作階段。

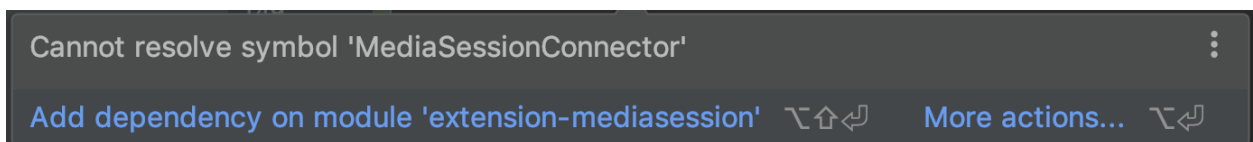
在類別頂端宣告下列兩個欄位。我們將在本節中持續使用這些欄位。

```
private MediaSessionCompat mediaSession;  
private MediaSessionConnector mediaSessionConnector;
```

您需要在「Module: demo」的模組層級 `build.gradle` 中新增「extension-mediasession」專案依附元件：

```
implementation project(path: ':extension-mediasession')
```

請注意，如果將滑鼠游標懸停在解析 `MediaSessionConnector` 時發生的錯誤上，Android Studio 可以協助您自動新增這項依附元件：



最後，加入下列內容，解決類別匯入問題：

```
import android.support.v4.media.session.MediaSessionCompat;  
import com.google.android.exoplayer2.ext.mediasession.MediaSessionConnector;
```

注意：在本程式碼研究室中，「媒體工作階段」一詞是指 `MediaSessionCompat` 的執行個體。這個類別來自 [media-compat 程式庫](#)，建議您使用這個程式庫編寫媒體應用程式。

建立活動時，我們會想建立媒體工作階段和媒體工作階段連接器，做為媒體工作階段和 `ExoPlayer` 之間的媒介。

插入這個函式的理想位置，也是建立 `ExoPlayer` 的位置。在我們的試用版應用程式中，我們可以將程式碼附加到 `initializePlayer()` 的結尾。請務必在玩家例項化後加入這項邏輯！

```
private void initializePlayer() {
    if (player == null) {
        ...
        player = ...
        ...
        mediaSession = new MediaSessionCompat(this, "sample");
        mediaSessionConnector = new MediaSessionConnector(mediaSession);
        mediaSessionConnector.setPlayer(player);
    }
    ...
}
```

釋放媒體工作階段

不再需要媒體工作階段時，請釋放該工作階段。在 `releasePlayer()` 中發布 `ExoPlayer` 時，我們也可以加入下列程式碼：

```
private void releasePlayer() {
    if (mediaSession != null) {
        mediaSession.release();
    }
    ...
}
```

管理媒體工作階段狀態

現在我們已例項化媒體工作階段，接下來需要確保在使用者與活動互動時，媒體工作階段的狀態會正確反映。

使用者啟動活動時，媒體工作階段應會啟動：

```
@Override
public void onStart() {
    ...
    if (mediaSession != null) {
        mediaSession.setActive(true);
    }
}
```

由於我們的應用程式不會在背景播放媒體，因此請務必確保媒體工作階段會在使用者離開活動時停用：

```
@Override
public void onStop() {
    super.onStop();
    if (mediaSession != null) {
        mediaSession.setActive(false);
    }
    ...
}
```

注意：如果未將媒體工作階段設為非使用中，可能會導致使用者獲得不想要的結果。舉例來說，如果使用者結束 Activity 時釋出 ExoPlayer，使用者螢幕鎖定畫面上可能仍會顯示無法操作的播放控制項。

注意：如要進一步瞭解媒體遙控器和媒體工作階段如何搭配運作，請參閱[媒體應用程式架構總覽指南](#)。

我們來執行試用版

1. 連上 Android 裝置或啟動模擬器。
2. 確認已選取「demo」，可從 Android Studio 工具列執行。



3. 按一下 Android Studio 工具列中的

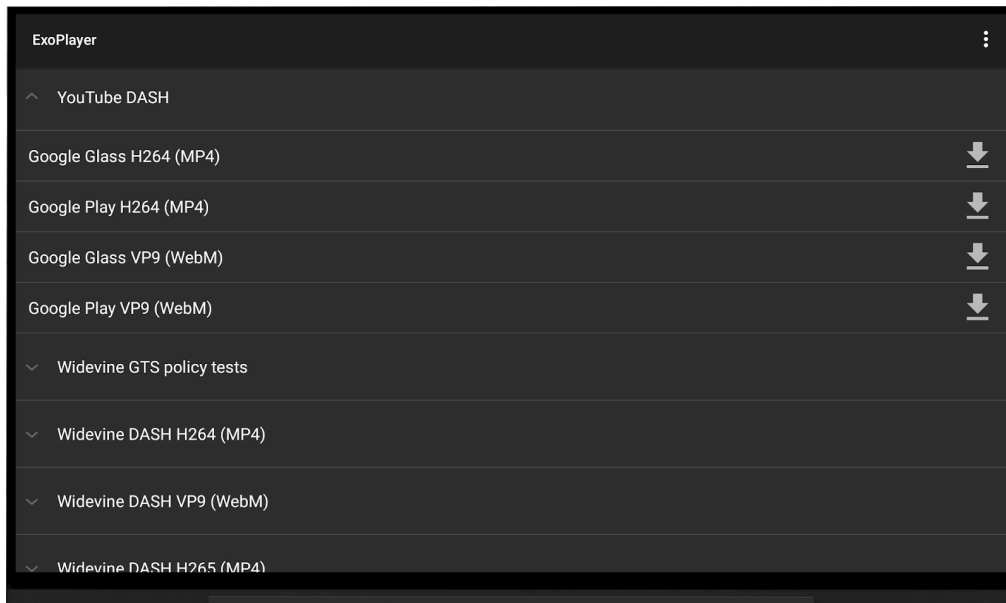


4. 應用程式在裝置上啟動後，選取要播放的影片串流。
5. 開始播放後，請使用下列 `adb` 指令控制媒體工作階段：

```
adb shell media dispatch pause
adb shell media dispatch play
adb shell media dispatch play-pause
adb shell media dispatch fast-forward
adb shell media dispatch rewind
```

注意：如果裝置不支援這些指令，請嘗試使用模擬器。

6. 此外，您也可以瞭解 Android 如何看待媒體工作階段。具體來說，您可以查看動作欄位，瞭解支援哪些動作。您在這裡看到的數字是動作 ID 的組合，如 [PlaybackState](#) 物件中所宣告。如要查看媒體工作階段的執行情況，請執行 `adb shell dumpsys media_session`
7. 如果使用附有麥克風的實體裝置，請嘗試喚醒 Google 助理並下達語音指令，例如「暫停」、「繼續」、「快轉 1 分鐘。」



在 *Android TV* 上執行的 *ExoPlayer* 範例。

注意：透過 Google 助理下達語音指令可能只適用於 *Android TV*，或是媒體在背景繼續播放且媒體工作階段保持啟用狀態時。

4. 包括播放佇列中項目的中繼資料

現在，我們可以擴充媒體工作階段支援的功能，先前我們已在 `initializePlayer()` 中建立 `MediaSessionConnector`。

新增 `TimelineQueueNavigator`

`ExoPlayer` 會將媒體結構表示為時間軸。如要進一步瞭解運作方式，請花點時間閱讀 `ExoPlayer` 的[時間軸物件](#)。輕觸這個結構，我們就能在內容變更時收到通知，並在收到要求時顯示目前播放內容的中繼資料。

為此，我們將定義 `TimelineQueueNavigator`。在 `initializePlayer()` 中找出 `MediaSessionConnector` 的例項，並在 `mediaSession` 初始化後加入 `TimelineQueueNavigator` 的實作內容。

```
mediaSessionConnector.setQueueNavigator(new TimelineQueueNavigator(mediaSession) {
    @Override
    public MediaDescriptionCompat getMediaDescription(Player player, int windowIndex) {
        return new MediaDescriptionCompat.Builder()
            .setTitle(player.getCurrentMediaItem().mediaMetadata.title)
            .setDescription("MediaDescription description for " + windowIndex)
            .setSubtitle("MediaDescription subtitle")
            .build();
    }
});
```

注意：如果這裡提供的標題為空值，Google 助理就無法從 `MediaDescription` 擷取這項資訊。

新增下列項目，解決類別匯入問題：

```
import android.support.v4.media.MediaDescriptionCompat;
import com.google.android.exoplayer2.ext.mediasession.TimelineQueueNavigator;
```

請注意，`windowIndex` 參數會對應至播放佇列中該索引的項目。

注意：這個佇列的內容取決於播放器的媒體來源。在提供的示範中，可以從範例選取活動中選取播放清單範例，該活動使用 `ConcatenatingMediaSource`。

新增中繼資料後，你可以測試 Google 助理是否能瞭解播放內容。在 Android TV 上播放影片時，啟動 Google 助理並詢問「現在播放的是什麼？」。

playWhenReady:false playbackState:ready window:0
video/avc(id:137 r:1920x1080 sib:0 sb:0 rb:415 db:0 mcd:0 dk:0)
audio/mp4a-latm(id:140 hz:44100 ch:2 sib:0 sb:0 rb:760 db:0 mcd:0 dk:0)

SELECT TRACKS



This is MediaDescription title.



Press the mic button to speak

步驟 5 · 約 5 分鐘

5. 自訂動作

也許播放器不支援某些動作，或是您想加入更多支援動作？現在讓我們深入瞭解媒體工作階段，我們先前已在 `initializePlayer()` 中建立 `MediaSessionConnector`。

宣告支援的動作

嘗試使用 `mediaSessionConnector.setEnabledPlaybackActions()` 自訂媒體工作階段要支援的動作。

請注意，完整組合如下：

```
mediaSessionConnector.setEnabledPlaybackActions(  
    PlaybackStateCompat.ACTION_PLAY_PAUSE  
        | PlaybackStateCompat.ACTION_PLAY  
        | PlaybackStateCompat.ACTION_PAUSE  
        | PlaybackStateCompat.ACTION_SEEK_TO  
        | PlaybackStateCompat.ACTION_FAST_FORWARD  
        | PlaybackStateCompat.ACTION_REWIND  
        | PlaybackStateCompat.ACTION_STOP  
        | PlaybackStateCompat.ACTION_SET_REPEAT_MODE  
        | PlaybackStateCompat.ACTION_SET_SHUFFLE_MODE  
);
```

讓我們再次看看平台如何公開這項資料：

1. 如先前所述，開始播放影片。
2. 執行 `adb shell dumpsys media_session`，瞭解 Android 如何查看媒體工作階段的中繼資料。
3. 找出包含中繼資料的行，並觀察標題和說明是否已納入並與 `com.google.android.exoplayer2.demo/sample` 建立關聯。

新增其他動作

我們可以透過一些額外動作擴展媒體工作階段。在本節中，我們只會新增字幕支援。

支援字幕

為媒體工作階段新增字幕支援功能後，使用者就能透過語音切換字幕。在您初始化媒體工作階段連接器的地方，新增下列項目：

```
mediaSessionConnector.setCaptionCallback(new MediaSessionConnector.CaptionCallback() {  
    @Override  
    public void onSetCaptioningEnabled(Player player, boolean enabled) {  
        Log.d("MediaSession", "onSetCaptioningEnabled: enabled=" + enabled);  
    }  
  
    @Override  
    public boolean hasCaptions(Player player) {  
        return true;  
    }  
  
    @Override  
    public boolean onCommand(Player player, ControlDispatcher controlDispatcher, String  
command, Bundle extras, ResultReceiver cb) {  
        return false;  
    }  
}  
);
```

最後，請修正所有缺少的匯入內容。

如要測試，請在 Android TV 上啟動 Google 助理，然後說出「啟用字幕」。檢查 Logcat 中的訊息，瞭解這項呼叫如何進入您的程式碼。

注意：如果從 `hasCaptions()` 傳回 `false`，Google 助理會通知使用者不支援字幕。

6. 恭喜

恭喜，您已成功將媒體工作階段新增至範例！

您已透過下列方式取得大量功能：

- 新增媒體工作階段，
- 將媒體工作階段連結至 ExoPlayer 執行個體，
- 新增中繼資料和其他動作。

現在您已瞭解豐富媒體應用程式內容，為使用者提供更多元體驗的關鍵步驟！

最後提醒

本程式碼研究室是以 ExoPlayer 原始碼的範例為基礎建構而成。您不需要從來源使用 ExoPlayer，建議改為提取 ExoPlayer 和 MediaSessionConnector 的依附元件，這樣就能更輕鬆地掌握最新版本。

如要這麼做，只要取代專案依附元件即可，例如：

```
implementation project(modulePrefix + 'library-core')
implementation project(path: ':extension-mediasession')
```

從 Maven 存放區提取，例如：

```
implementation 'com.google.android.exoplayer:exoplayer-core:2.+'
implementation 'com.google.android.exoplayer:extension-mediasession:2.+'
```

參考文件

- [使用媒體工作階段](#)
 - [媒體應用程式架構總覽](#)
 - [ExoPlayer 的時間軸結構](#)
 - [ExoPlayer 開發人員指南](#)
 - [ExoPlayer 的 MediaSessionConnector API 參考資料](#)
-